

UNITAT 5 · SETMANA 7

Canonades i composició de processos

📅 Setmana 7 · dates per confirmar </> Laboratori TD4 (result + repte IA) ~ 2 animacions

MANUAL M5 · TEORIA DE LA UNITAT

Canonades i composició de processos

Lectura bàsica de la unitat 5 (≈ 1 h). Les sortides són reals: programes del TD4 executats a la màquina del curs.

CONTINGUT

- | | | | |
|----|---|----|--|
| §1 | La canonada: una memòria intermèdia amb dos extrems | §3 | Redirigir amb <code>dup2</code> i executar |
| §2 | Les regles: bloqueig, final de fitxer i SIGPIPE | §4 | Pipelines de N ordres |
| | | §5 | Més enllà |

§1 La canonada: una memòria intermèdia amb dos extrems

`pipe(fds)` crea una memòria intermèdia circular al nucli i retorna dos descriptors: `fds[0]`, l'extrem de **lectura**, i `fds[1]`, l'extrem d'**escriptura**. El que s'escriu en un es llegeix de l'altre, en ordre, com un flux de bytes sense límits de missatge. La canonada no té nom (a diferència d'una FIFO, unitat 3): només la poden fer servir els processos que hereten els seus descriptors, i per això es crea **abans** de `fork`. Després, cada procés **tanca l'extrem que no fa servir**: el pare que escriu tanca `fds[0]`, el fill que llegeix tanca `fds[1]`.

C tds/td4/mypipe.c

```
/*
 * TD4 - exercises 1 and 2: an unnamed pipe between a parent (writer) and a child
 * (reader).
 * ./mypipe [count]    the parent writes "Hello World\n" `count` times (default 10)
 * The child reads AT MOST 500 bytes, once, prints them and leaves.
 *
 * Exercise 2: with a large count (10000 x 12 bytes = 120 KB > 64 KiB, the pipe
 * capacity)
 * the parent blocks on write() once the pipe is full. When the child exits, nobody can
 * read
 * any more: the next write() raises SIGPIPE and kills the parent ("Broken pipe", status
```

```

141
* in the shell). That only works if the parent CLOSED its own read end: with
* -DKEEP_READ_END (Makefile target mypipe_bug) the parent itself is still a potential
reader,
* the kernel never raises SIGPIPE and the parent blocks forever.
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define MESSAGE    "Hello World\n"
#define READ_MAX   500

int main(int argc, char *argv[])
{
    int count = argc > 1 ? atoi(argv[1]) : 10;
    int fds[2];                                /* fds[0] read end, fds[1] write
end */

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                             /* child: reader */
        exit_if(close(fds[1]) == -1, "close");    /* never writes */
        char buffer[READ_MAX];
        ssize_t n = read(fds[0], buffer, READ_MAX); /* one single read */
        exit_if(n == -1, "read");
        exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
        fprintf(stderr, "I am the child, I read %zd bytes and I leave\n", n);
        exit_if(close(fds[0]) == -1, "close");
        return EXIT_SUCCESS;
    }

    /* parent: writer */
#ifdef KEEP_READ_END
    exit_if(close(fds[0]) == -1, "close");        /* THE line that exercise 2 is
about */
#endif
    for (int i = 0; i < count; i++)
        exit_if(write_all(fds[1], MESSAGE, strlen(MESSAGE)) == -1, "write");

```

```
fprintf(stderr, "I am the parent, I wrote %d messages and I leave\n", count);
exit_if(close(fds[1]) == -1, "close");
exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");
return EXIT_SUCCESS;
}
```

Sortida real

```
$ ./mypipe
I am the parent, I wrote 10 messages and I leave
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
I am the child, I read 120 bytes and I leave
```

Practica-ho → TD4 · 1

§2 Les regles: bloqueig, final de fitxer i SIGPIPE

- **Llegir** d'una canonada buida es bloqueja fins que arriben dades... tret que no quedi **cap** descriptor d'escriptura obert en cap procés: llavors `read` retorna 0, final de fitxer.
- **Escriure** en una canonada plena es bloqueja fins que algú llegeix. La capacitat per defecte a Linux és de 64 KiB (65.536 bytes); es consulta i es canvia amb `fcntl(F_GETPIPE_SZ / F_SETPIPE_SZ)`.
- Escriure quan no queda **cap lector** provoca el senyal `SIGPIPE`, que per defecte mata el procés (estat 141 = 128 + 13 al shell); si s'ignora, `write` falla amb `EPIPE`.
- Una escriptura de fins a `PIPE_BUF` = 4096 bytes és **atòmica**: no s'intercala amb les d'altres escriptors.

El TD4 · 2 mostra les tres primeres regles alhora. El fill llegeix 500 bytes una vegada i acaba; el pare escriu N missatges de 12 bytes. Si el pare acaba abans que el fill se'n vagi, tot va bé; si el fill se'n va primer, el `write` següent del pare rep `SIGPIPE`. Amb molts missatges passa sempre, però entremig és una **cursa** entre dos processos que s'executen en nuclis diferents: el resultat canvia d'una execució a una altra. Mesurat 30 vegades per mida:

Sortida real

```
$ for n in 50 100 ... 1000; do run ./mypipe n 30 times; count exit statuses; done
 50 messages: 30 x status 0      0 x status 141 (SIGPIPE)
100 messages: 30 x status 0      0 x status 141 (SIGPIPE)
200 messages: 28 x status 0      2 x status 141 (SIGPIPE)
300 messages: 19 x status 0     11 x status 141 (SIGPIPE)
400 messages:  5 x status 0     25 x status 141 (SIGPIPE)
500 messages:  5 x status 0     25 x status 141 (SIGPIPE)
1000 messages:  0 x status 0     30 x status 141 (SIGPIPE)
```

Si el pare oblida tancar `fds[0]`, ell mateix compta com a lector: no rep mai `SIGPIPE`, omple els 64 KiB i es bloqueja per sempre esperant que «algú» llegeixi:

Sortida real

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 = killed by
timeout: it was blocked)"
I am the child, I read 500 bytes and I leave
exit status: 124 (124 = killed by timeout: it was blocked)
```

Practica-ho → A5.2 capacitat i cursa · TD4 · 2

§3 Redirigir amb `dup2` i executar

`dup2(fds[1], STDOUT_FILENO)` fa que el descriptor 1 apunti a la canonada (unitat 3: copia el punter a l'entrada de la taula de fitxers oberts). Com que `exec` conserva els descriptors (unitat 4), el programa que s'executi després escriurà a la canonada sense saber-ho. Així es connecten programes d'altres, com `ps`, a un lector nostre:

C tds/td4/upper.c

```
int main(void)
{
    int fds[2];

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                /* child */
        exit_if(close(fds[1]) == -1, "close"); /* otherwise it never sees end of file */
        to_upper(fds[0], STDOUT_FILENO);
        return EXIT_SUCCESS;
    }

    /* parent */
    exit_if(close(fds[0]) == -1, "close");
    exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
    exit_if(close(fds[1]) == -1, "close"); /* 1 already refers to the pipe */
    execlp("ps", "ps", (char *)NULL);      /* "ps" twice: file to run, then
    argv[0] */
    exit_if(1, "execlp ps");
    return EXIT_FAILURE;
}
```

Sortida real

```
$ ./upper | head -4
PID TTY          TIME CMD
6706 ?           00:00:25 UVICORN
9771 ?           00:00:03 SYSTEMD
9801 ?           00:00:00 (SD-PAM)
```

Practica-ho → TD4 · 3

§4 Pipelines de N ordres

Per a $c_1 \mid c_2 \mid \dots \mid c_N$ el shell crea $N - 1$ canonades i N fills. L'algorisme cap en un bucle amb una variable `in` que guarda l'extrem de lectura de la canonada anterior: per a cada ordre, si no és l'última, `pipe`; `fork`; el fill fa `dup2(in, 0)` i `dup2(fds[1], 1)`, tanca tota la resta i crida `exec`; el pare tanca `in` i `fds[1]` i guarda `fds[0]` com a nou `in`. Al final els espera tots i retorna l'estat de l'última (a bash, `set -o pipefail` retorna el de l'última que ha fallat).

La regla d'or: **cada canonada ha d'acabar amb exactament un descriptor d'escriptura i un de lectura**, en els dos processos que la fan servir. Un sol `close` oblidat —al pare o en un fill que ha heretat la canonada— deixa un escriptor de més i l'última ordre espera un final de fitxer que no arriba mai.

C tds/td4/pipeline_args.c

```
int in = STDIN_FILENO;
pid_t last = -1;
for (int i = 0; i < ncommands; i++) {
    int fds[2] = { -1, -1 };
    if (i < ncommands - 1)
        exit_if(pipe(fds) == -1, "pipe");

    pid_t pid = fork();
    exit_if(pid == -1, "fork");
    if (pid == 0) {
        if (in != STDIN_FILENO) {
            exit_if(dup2(in, STDIN_FILENO) == -1, "dup2 stdin");
            exit_if(close(in) == -1, "close");
        }
        if (fds[1] != -1) {
            exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2 stdout");
            exit_if(close(fds[0]) == -1 || close(fds[1]) == -1, "close");
        }
        execvp(commands[i][0], commands[i]);
        perror(commands[i][0]);
        _exit(127);
    }
    if (in != STDIN_FILENO)
        exit_if(close(in) == -1, "close");
    if (fds[1] != -1) {
        exit_if(close(fds[1]) == -1, "close");
        in = fds[0];
    }
    last = pid;
}
```

Sortida real

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

Sortida real

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

Practica-ho → A5.1 construir un pipeline · TD4 · 4 i 5

§5 Més enllà

- `pipe2(fds, 0_CLOEXEC)` crea la canonada amb els descriptors marcats per tancar-se en fer `exec`: evita que un programa llançat hereti extrems que no li pertanyen (i que, com al §4, impedirien un final de fitxer).
- `splice` i `tee` mouen dades entre una canonada i un fitxer o sòcol dins del nucli, sense copiar-les a l'espai del procés.
- La **substitució de processos** de bash (`diff <(ls a) <(ls b)`): el shell crea canonades i les passa com a camins `/dev/fd/N`.
- La comunicació **bidireccional** necessita dues canonades o un `socketpair` (unitat 8).